

Sistemi Operativi

Relazione di Progetto

a.a. 2023/2024

Gaetano Turone¹ and Federico Zerbin²

¹gaetano.turone@edu.unito.it, matr: 1045777

²federico.zerbin@edu.unito.it, matr: 902129

1 Progettazione e struttura di base

Abbiamo deciso di lavorare utilizzando lo standard POSIX. La struttura del progetto si articola in moduli ciascuno rappresentante un tipo di processo, con l'aggiunta di un makefile e due directory, una per i file common e una per la configurazione. Nel dettaglio

- ogni modulo per processo avrà un file .c, un header e un makefile proprio.
- Nella cartella common è presente un header globale incluso in tutti i processi.
- Il makefile nella root, utilizza le regole dei singoli makefiles dei moduli per compilare tutto in un'unica volta dalla root con il comando “make all”. Questo genera un eseguibile all'interno della subdirectory /master.

Infine il programma è provvisto di 4 file di configurazione .txt letti a runtime ed inseriti nella cartella config, ciascuno pensato per un tipo di terminazione specifico, nel dettaglio

- config1.txt provoca la terminazione per timeout.
- config2.txt provoca la terminazione per explode.
- config3.txt provoca la terminazione per blackout.
- config4.txt provoca la terminazione per meltdown.

2 Scelte implementative generali

Su un piano generico rispetto agli IPC impiegati e i meccanismi di base:

- Abbiamo scelto di utilizzare memorie condivise per mantenere pid, statistiche e parametri di configurazione, sincronizzandole con un mutex dedicato per ciascuna.
- L'attivazione di un atomo è ottenuta con l'invio da parte dell'attivatore ad un atomo casuale del segnale SIGUSR1 opportunamente rinominato e gestito.
- L'attivazione o disattivazione del processo inibitore da master l'abbiamo strutturata in modo da essere una routine di gestione sostitutiva del segnale SIGTSTP (CTRL + Z generabile quindi da tastiera a runtime).
- Inibitore utilizza anche una coda di messaggi per leggere informazioni sui cambiamenti da lui introdotti (scissioni non avvenute o energia assorbita).

Per quanto riguarda la temporizzazione:

- tutti i processi ad azione periodica ad eccezione del master lavorano in nanosecondi.
- Per consentire questa precisione e per massimizzare la parallelizzazione delle azioni di master, attivatore e alimentazione abbiamo utilizzato i timer POSIX impostando la loro sigevent con la flag SIGEV_THREAD. In questo modo ogni azione periodica è relegata in un blocco di codice compatto che viene eseguito da un singolo nuovo thread.
- La partenza sincronizzata dei processi è ottenuta attraverso l'uso combinato di due semafori contatore gestiti dal master.

3 Implementazione dei moduli

Scendendo un poco più nel dettaglio rispetto ad ogni modulo questa è un'idea di come i processi funzionano:

3.1 Processo Master

Abbiamo deciso di implementare il processo master utilizzando un timer che scatta ogni secondo. Come metodo di notifica ha SIGEV_THREAD che crea un nuovo thread ed esegue la funzione di gestione della simulazione. Questa tiene conto dei secondi passati, ogni secondo controlla se c'è stata una terminazione, stampa le statistiche e preleva l'energia (ENERGY DEMAND). La funzione si appoggia a due memorie condivise, la prima tiene in memoria le statistiche da stampare ogni secondo, la seconda tiene i parametri di configurazione. Ciascuna di queste memorie è protetta da un proprio semaforo MUTEX. All'avvio, il processo master carica la configurazione scelta, inizializza gli handler per i segnali, il timer, crea i processi figli (atomo, attivatore e alimentazione) tramite fork() e salva i loro PID in un'altra shared memory protetta da un semaforo. Abbiamo gestito la partenza sincronizzata dei processi con due semafori. una volta creati tutti i figli il master si mette in attesa sul semaforo "all ready", ogni figlio appena è pronto sblocca questo semaforo e attende poi il segnale di partenza sul semaforo "all go". Una volta pronti tutti il master rilascia il semaforo all go, la simulazione parte, entra in ciclo while "infinito" ed uscirà quando i secondi passati raggiungono i secondi impostati nella configurazione (SIM DURATION) oppure nel caso avvenga una delle altre terminazioni. Alla terminazione, viene rimosso il timer, vengono uccisi i processi figli, viene stampato il tipo di terminazione e vengono deallocati gli oggetti IPC.

3.2 Processo Atomo

Abbiamo realizzato il processo atomo, come un processo che cicla fino a quando non riceve un segnale di terminazione oppure fino a quando non viene inserito nelle scorie. Abbiamo deciso di implementare la scelta del numero atomico casuale con un seed randomico utilizzando i nano-secondi per una maggiore randomicità (Abbiamo preso spunto dal codice del Prof. Bini nel file test-sem-cook.c). Per gestire la scissione degli atomi, abbiamo ridefinito il segnale SIGUSR1 in ACTIVATE_ATOM_SIGNAL che viene gestito dall'handler dei segnali del processo atomo: quando un processo atomo riceve questo segnale, viene chiamata la funzione divide(), che controlla se l'atomo è esausto, se lo è lo inserisce nelle scorie altrimenti esegue la fork, preparando prima il numero atomico del figlio in modo casuale e libera l'energia. Quando esegue la fork, salva il PID del figlio creato nella memoria condivisa dei PID.

3.3 Processo Attivatore

Come per il master, il processo attivatore ha un timer che crea periodicamente thread a cui viene associata la funzione di richiesta di attivazione, che manderà il segnale ACTIVATE_ATOM_SIGNAL ad un atomo. L'atomo viene scelto in maniera casuale dalla memoria condivisa dei pid tramite la funzione rand(), dopodiché aggiorna il numero di attivazioni totali.

3.4 Processo Alimentazione

Ogni STEP ALIMENTAZIONE nanosecondi il timer chiama la funzione di alimentazione periodica, che va ad eseguire `fork()` N NEW ATOMS volte e salva i pid dei nuovi atomi nella shm dei pid.

3.5 Processo Inibitore

Il processo inibitore viene attivato da tastiera a runtime tramite CTRL+Z. Abbiamo ridefinito il segnale SIGTSTP per gestire l'attivazione e lo spegnimento del processo inibitore. Il master possiede un handler di segnale apposito che attiva e disattiva l'inibitore. Quando è attivo, fa da tramite per gli atomi e per l'attivatore perché ne condiziona i meccanismi. In particolare se è attivo:

- l'attivatore pesca un numero compreso tra 0 e 10 e pesca il pid casuale solo se il numero estratto è pari, altrimenti la scissione non avviene e notifica l'inibitore tramite l'invio di un messaggio, gestito con una coda di messaggi. All'interno del messaggio è contenuto un counter che tiene traccia delle scissioni bloccate fino a quel momento. L'inibitore si occupa di stampare solo quante scissioni sono state impedito.
- L'atomo invece libera meno energia. L'energia liberata viene divisa a metà, una metà viene conteggiata come energia liberata normalmente e l'altra metà viene inviata all'inibitore tramite la coda di messaggi. in questo caso l'inibitore la salva come energia inibita da stampare nelle statistiche.